Streaming Calibration Weighting*

Gaurav Sood†

March 27, 2026

Abstract

Calibration weighting adjusts observation weights so that the weighted distribution of categorical covariates matches known population marginals. The problem arises in survey research (post-stratification, raking), causal inference (covariate balancing), and domain adaptation (importance weighting to a target distribution). The standard approach operates on the complete sample and must be rerun whenever new observations arrive. We develop two streaming algorithms that update weights incrementally as each observation is processed. By maintaining running sufficient statistics (cumulative weighted marginal totals), each update requires $O(Kp)$ work, where K is the number of inner gradient steps and p is the number of calibration categories, regardless of how many observations have been processed. The first algorithm performs projected gradient descent on a squared marginal loss; the second performs multiplicative updates that parallel the structure of iterative proportional fitting. On a fixed sample with feasible targets, both methods converge to weight vectors satisfying the marginal constraints. In streaming simulations with drifting covariate distributions, the online methods track population marginals throughout data collection. Periodic batch recalibration at modest intervals (every 50–100 observations) offers a practical middle ground, matching batch accuracy with incremental availability.

1 Introduction

Calibration weighting is the problem of finding observation weights so that the weighted covariate distribution matches a known target distribution. The problem is common across applied statistics. In survey research, raking and post-stratification adjust respondent weights so that weighted demographic totals match census benchmarks (Deming and Stephan, 1940; Deville and Särndal, 1992). In causal inference, entropy balancing and covariate balancing propensity scores find weights that equalize covariate distributions between treatment and control groups (Hainmueller, 2012; Imai and Ratkovic, 2014). In domain adaptation, importance weights correct for distributional shift between source and target domains.

*Replication materials: https://github.com/finite-sample/onlinerake.

†Gaurav can be reached at gsood07@gmail.com

In each of these settings, the structure is the same. There are n observations, each with a vector of categorical covariates (e.g., age group, sex, region). There are population marginals specifying what proportion of the weighted sample should fall into each category. The task is to find positive weights, as close to the base weights as possible, such that the weighted category proportions match the targets. This is what we mean by "calibration" throughout the paper. We are not addressing probability calibration (mapping classifier scores to well-calibrated probabilities, as in Platt scaling or isotonic regression), which is a different problem despite sharing the name.

The dominant algorithm for this problem is iterative proportional fitting (IPF, also known as raking; Deming and Stephan, 1940). IPF cycles through the calibration variables, multiplicatively adjusting all weights so that the weighted marginal for each variable matches its target. When the marginal constraints are feasible, the algorithm converges to a weight vector that minimizes Kullback–Leibler divergence from the base weights subject to the constraints (Csiszár, 1975). This KL minimizer is unique whenever it exists. Other algorithms for the same problem include entropy balancing (Hainmueller, 2012), which solves a closely related problem via an unconstrained dual formulation, and the generalized calibration framework of Deville and Särndal (1992), which allows different divergence measures between calibrated and base weights.

All of these are batch algorithms: they require the full sample and must be rerun whenever new observations arrive or targets change. This paper develops streaming alternatives. The motivation is settings where observations arrive sequentially and weighted estimates are needed throughout data collection, not only after the sample is complete. Rolling surveys, real-time dashboards during panel data collection, and online systems that must maintain calibrated weights as users arrive are examples. Batch recalibration after each new observation costs $O(np)$ per update, where n is the current sample size and p is the number of calibration categories. We want algorithms where the per-observation cost does not grow with n.

We make three contributions. First, we formulate calibration as minimizing a squared marginal loss and derive two per-observation update rules: an additive (SGD) update and a multiplicative (MWU) update (Section 3). Both operate on the new observation's weight using only running sufficient statistics, achieving $O(Kp)$ cost per observation. Second, we compare accuracy, weight efficiency, and computational cost against batch IPF and periodic recalibration in simulations with stationary and drifting covariate distributions (Section 4). Third, we discuss the convergence properties of the streaming algorithms, their relationship to IPF, and the structural asymmetry between early and late observations (Section 5).

Two clarifications about scope. First, throughout the paper "raking" and "IPF" refer specifically to the batch multiplicative algorithm of Deming and Stephan (1940). "Calibration" refers to the general problem of matching weighted marginals to targets. Our streaming methods minimize squared marginal error, which differs from the KL objective that IPF minimizes. On a feasible problem with a unique solution, the objectives agree; when multiple solutions exist (as in typical settings where $n \gg p$), different divergence measures select different weight vectors. Second, this paper takes a purely design-based approach: the weights are chosen to match known marginals without modeling the relationship between the covariates and any outcome of interest. Model-assisted calibration, as in MRP (Gelman and Little, 1997) or the GREG estimator (Deville and Särndal, 1992), uses such a model

to improve efficiency. Extending the streaming framework to model-assisted settings is a natural direction but outside our current scope.

2 Background

2.1 The Calibration Problem

Let $x_i \in \{0,1\}^p$ denote the vector of category indicators for observation i. Each indicator marks membership in a calibration category: for example, $x_{i1} = 1$ if observation i is female, $x_{i2} = 1$ if under 30, and so on. Let $t \in (0,1)^p$ denote the target proportions for each category. The calibration problem is to find positive weights $w_1, \ldots, w_n$ such that the weighted category proportions match the targets:

$$m_j(\mathbf{w}) := \frac{\sum_{i=1}^n w_i x_{ij}}{\sum_{i=1}^n w_i} = t_j, \quad j = 1, \ldots, p. \tag{1}$$

When multiple weight vectors satisfy (1), one selects among them by minimizing a divergence from base weights u_i (often uniform, design weights in a survey context, or inverse propensity weights in a causal inference context). The choice of divergence measure defines the calibration method: KL divergence yields raking/entropy balancing, chi-squared distance yields the GREG estimator, and other distances yield the generalized calibration estimators of Deville and Särndal (1992).

2.2 Iterative Proportional Fitting

IPF solves the calibration problem by cycling through the p categories. For category j, every weight is scaled by the ratio of the target marginal to the current weighted marginal:

$$w_i \leftarrow w_i \cdot \frac{t_j}{m_j(\mathbf{w})} \quad \text{for all } i \text{ with } x_{ij} = 1.$$

After processing category j, the weighted marginal for j equals t_j exactly (though earlier marginals may be disturbed). The algorithm cycles through all categories until all marginals converge. IPF converges whenever the marginal constraints are feasible and produces the weight vector minimizing $\mathrm{KL}(\mathbf{w}\|\mathbf{u})$ subject to (1) (Csiszár, 1975).

IPF requires access to the full weight vector at each step. Adding a single new observation requires rerunning the entire algorithm, at cost $O(np)$ per IPF iteration times the number of cycles to convergence. For large n or frequent updates, this cost motivates streaming alternatives.

2.3 Related Work

The calibration framework of Deville and Särndal (1992) and Deville et al. (1993) provides the theoretical foundation for weight adjustment to known totals in the survey sampling literature. Raking (IPF applied to demographic margins) is the standard implementation; Kolenikov and Hammer (2015) reviews recent practice. Entropy balancing (Hainmueller,

2012) solves a dual formulation of the same KL minimization problem that IPF addresses. Where IPF works in the primal (multiplicatively adjusting weights, cycling through constraints), entropy balancing works in the dual (finding Lagrange multipliers for the moment constraints via Newton-type methods, then recovering weights through exponential tilting). The two approaches produce the same weights when both converge, but differ in computational profile: IPF is simple and scales well with n; entropy balancing handles many simultaneous constraints more naturally. Zhao and Percival (2017) establishes double robustness of the entropy balancing estimator. Zubizarreta (2015) studies calibration with explicit weight bounds.

None of these methods are designed for streaming updates. In machine learning, online calibration of probability forecasts has been studied via Platt scaling (Platt, 1999), isotonic regression (Zadrozny and Elkan, 2002), and temperature scaling (Guo et al., 2017), but these address a different problem (mapping continuous scores to calibrated probabilities) and are trained in batch. The multiplicative weights update method (Arora et al., 2012) and mirror descent with entropic regularization (Beck and Teboulle, 2003) provide the algorithmic template for our MWU update. Online convex optimization (Hazan, 2016) provides the convergence framework, though our setting differs from the standard one because the problem dimension grows with the data.

3 Streaming Calibration

3.1 Objective

We formulate calibration as minimizing the squared marginal error:

$$L(\mathbf{w}) = \frac{1}{2}\sum_{j=1}^{p}\big(m_j(\mathbf{w}) - t_j\big)^2, \tag{2}$$

subject to $w_i \in [\varepsilon, M]$ for all i, where $\varepsilon > 0$ and $M < \infty$ prevent degenerate weights. When the margin constraints (1) are feasible within these bounds, any minimizer of L satisfies them exactly ($L = 0$).

Because the margins m_j are ratios of linear functions of $\mathbf{w}$, the loss L is not globally convex. However, it has a simple structure: $L \geq 0$ everywhere, $L = 0$ exactly at feasible weight vectors, and $\nabla L(\mathbf{w}) = 0$ implies either $L(\mathbf{w}) = 0$ (feasibility achieved) or a degenerate saddle point that gradient descent escapes under generic conditions. In our experiments, gradient descent on L converges reliably to feasible solutions when they exist. Under a normalization that fixes $\sum_i w_i$, the marginals become linear in $\mathbf{w}$ and L becomes a convex quadratic, which provides formal rate guarantees; we discuss this further in Section 5.

This objective differs from the KL divergence minimized by IPF. In typical settings, $n \gg p$ and the calibration problem is underdetermined: many weight vectors satisfy the marginal constraints. Squared-error calibration selects the feasible weights closest to the initialization in a sense determined by the loss geometry, while KL calibration (IPF) selects those closest to the base weights in relative-entropy terms. When the solution is unique, the two objectives agree.

3.2 Sufficient Statistics

The key observation enabling streaming updates is that the margin $m_j(\mathbf{w})$ depends on the data only through two running totals:

$$S_j^{(t)} = \sum_{i=1}^{t} w_i x_{ij}, \qquad W^{(t)} = \sum_{i=1}^{t} w_i, \tag{3}$$

so that $m_j(\mathbf{w}) = S_j^{(t)}/W^{(t)}$. When a new observation $x^{(t+1)}$ arrives with initial weight w_{t+1} (set to 1 for uniform base weights, or to the design weight u_{t+1} when base weights are available), the sufficient statistics update as

$$S_j^{(t+1)} = S_j^{(t)} + w_{t+1}\, x_{(t+1),j}, \qquad W^{(t+1)} = W^{(t)} + w_{t+1},$$

and the current margins are immediately available without scanning the full dataset. The gradient of L with respect to the new observation's weight is also computable from these statistics, as we show next.

3.3 Gradient

The gradient of L with respect to weight w_k is

$$\frac{\partial L}{\partial w_k} = \frac{1}{W} \sum_{j=1}^{p} (m_j - t_j)(x_{kj} - m_j), \tag{4}$$

where $W = \sum_i w_i$ and $m_j = S_j/W$. This follows from the quotient rule applied to $m_j(\mathbf{w}) = S_j/W$:

$$\frac{\partial m_j}{\partial w_k} = \frac{x_{kj}W - S_j}{W^2} = \frac{x_{kj} - m_j}{W}.$$

The gradient has a natural interpretation. Because the SGD update subtracts η times the gradient, observation k's weight increases when its indicators x_{kj} align with underrepresented margins ($m_j < t_j$, so the gradient contribution is negative) and decreases when they align with overrepresented margins ($m_j > t_j$, so the gradient contribution is positive). The algorithm upweights observations that look like the underrepresented population segments and downweights those that look like overrepresented segments.

Computing (4) for a single observation k requires $O(p)$ work given the sufficient statistics S_j and W, which are maintained incrementally.

3.4 Algorithm 1: Additive Update (SGD)

When observation $t+1$ arrives:

1. Initialize $w_{t+1} = 1$.
2. Update sufficient statistics: $S_j \leftarrow S_j + x_{(t+1),j}$ for all j; $W \leftarrow W + 1$.

3. For $k = 1, \ldots, K$ inner steps:
   (a) Compute the gradient (4) for observation $t+1$.
   (b) Update $w_{t+1} \leftarrow \mathrm{clip}(w_{t+1} - \eta\, \nabla_{t+1} L,\ \varepsilon, M)$.
   (c) Update sufficient statistics to reflect the weight change: $\Delta w = w_{t+1}^{\text{new}} - w_{t+1}^{\text{old}}$; $S_j \leftarrow S_j + \Delta w \cdot x_{(t+1),j}$; $W \leftarrow W + \Delta w$.

Each inner step costs $O(p)$, so each new observation costs $O(Kp)$ regardless of n. The projection (clipping) enforces the weight bounds. Because the margins $m_j = S_j/W$ and the gradient (4) depend only on weight ratios, the algorithm is scale-invariant and requires no explicit normalization.

The central design choice is that the inner loop adjusts only the new observation's weight, treating all previous weights as fixed. This is what makes the algorithm streaming: the cost of processing each arrival is independent of how many observations preceded it. The choice is justified by the observation that each new arrival perturbs the current margins by $O(1/t)$, so the previous weights, which were approximately calibrated when they were set, remain approximately calibrated afterward. The K inner gradient steps on the new weight compensate for this small perturbation rather than recalibrating from scratch.

3.5 Algorithm 2: Multiplicative Update (MWU)

The multiplicative variant replaces the additive step with an exponentiated gradient:

$$w_{t+1} \leftarrow \mathrm{clip}\Big(w_{t+1} \cdot \exp\big(-\eta\, \nabla_{t+1} L\big),\ \varepsilon, M\Big),$$

with the same sufficient-statistics bookkeeping. This update ensures positivity without clipping (the clip enforces only the upper bound M) and resembles the multiplicative structure of IPF.

4 Experiments

4.1 Setup

We compare five calibration strategies across six scenarios.

Algorithms. (1) *Batch IPF*: classical raking on the full sample, the accuracy benchmark. (2) *Streaming SGD (fixed η)*: Algorithm 1 with learning rate 5.0 and $K = 3$ inner steps. (3) *Streaming SGD (Robbins-Monro)*: Algorithm 1 with $\eta_t = 5.0/t^{0.6}$ and $K = 3$. (4) *Streaming MWU*: Algorithm 2 with learning rate 1.0 and $K = 3$. (5) *Periodic IPF*: batch IPF rerun every 50 or 100 observations, providing incremental availability with batch accuracy.

Scenarios. *Standard* ($n = 3000$, $p = 8$, moderate target–sample discrepancy). *Large* ($n = 50{,}000$, $p = 8$). *Hard targets* ($n = 3000$, $p = 8$, extreme target proportions near 0.08 or 0.92). *High-dimensional* ($n = 3000$, $p = 50$). *Drift* ($n = 5000$, $p = 8$, covariate distribution shifts linearly over the stream). *Small easy* ($n = 500$, $p = 4$). We run 50 replications per scenario.

Table 1. Comparison of raking algorithms (mean ± SE over 50 seeds).

Scenario	Algorithm	Loss	ESS	Efficiency	Runtime (s)
Standard	Batch IPF	0.00	2314 ± 30	77.1%	0.006
	SGD (fixed LR)	9.4e-4	2165 ± 32	72.2%	0.86
	SGD (R-M)	0.052 ± 0.003	2960 ± 2	98.7%	0.87
	MWU	0.010 ± 0.001	2078 ± 59	69.3%	0.98
	Periodic IPF	0.00	2314 ± 30	77.1%	0.02
Large (n=50k)	Batch IPF	0.00	38750 ± 467	77.5%	0.08
	SGD (fixed LR)	8.7e-4	36598 ± 487	73.2%	97.1
	SGD (R-M)	0.060 ± 0.003	49939 ± 3	99.9%	94.1
	MWU	0.010 ± 0.001	31806 ± 1029	63.6%	119.0
	Periodic IPF	0.00	38750 ± 467	77.5%	1.5
Hard Targets	Batch IPF	0.00	41 ± 2	1.4%	0.003
	SGD (fixed LR)	0.014 ± 0.001	32 ± 3	1.1%	0.52
	SGD (R-M)	1.15 ± 0.01	2537 ± 7	84.6%	0.53
	MWU	0.23 ± 0.004	77 ± 2	2.6%	0.59
	Periodic IPF	0.00	41 ± 2	1.4%	0.02

Metrics. Final squared marginal loss (2). Effective sample size ESS $= (\sum w_i)^2 / \sum w_i^2$. Weight efficiency ESS$/n$. Runtime in seconds. For streaming methods, we also report the marginal error trajectory over the stream.

4.2 Results

Table 1 reports results for three representative scenarios.

Accuracy. Batch IPF achieves zero loss because the experimental targets are feasible by construction. Among streaming methods, SGD with fixed learning rate is closest (loss 0.0009 on standard, corresponding to root-mean-square marginal error of roughly $\sqrt{2 \times 0.0009/p}$ per category), followed by MWU (0.010) and SGD with Robbins-Monro (0.052). Periodic IPF at 100-observation intervals also achieves zero loss at its final recalibration.

Weight efficiency. Streaming SGD with Robbins-Monro preserves the highest weight efficiency (98.7%) but also has the highest loss (0.052) among the methods that converge. High efficiency with nonzero loss indicates that the algorithm has not fully calibrated: an algorithm that leaves all weights at one achieves 100% efficiency with no calibration at all. SGD with fixed η achieves 72.2%, and MWU 69.3%. Batch IPF efficiency is 77.1% on the standard scenario and depends entirely on how far the sample marginals are from the targets.

Computational cost. On the large scenario (n = 50,000), batch IPF completes in 0.08 seconds. Streaming SGD requires 97 seconds total across all observations. The per-observation cost is constant ($O(Kp)$), but the total cost over n observations is $O(nKp)$, compared to $O(npC)$ for batch IPF where C is the number of convergence cycles. With $K = 3$ inner

steps and C typically small (under 20), the total work is comparable; the 97-second runtime reflects Python overhead on 50,000 sequential calls rather than a fundamentally different workload. The streaming methods' advantage is latency (calibrated estimates at each step), not total throughput. Periodic IPF at 100-observation intervals achieves batch accuracy in 1.5 seconds total, combining low latency with low total cost.

Hard targets. When target proportions are extreme, achieving exact calibration forces most of the weight onto a small subset of observations. Batch IPF drives loss to zero but collapses ESS to 41 out of 3000 (1.4% efficiency). Streaming SGD with Robbins-Monro preserves 84.6% efficiency but its decaying step size prevents the weights from reaching the extreme values needed, stalling at loss 1.15. This reflects a tradeoff inherent in the Robbins-Monro schedule: convergence guarantees and weight stability come at the cost of slow adaptation when targets require extreme reweighting. The algorithm has failed to calibrate, but the implicit regularization may be desirable if the practitioner prefers stable weights to exact marginals. When exact calibration requires extreme weights, the practitioner must decide which matters more.

Drift. When the covariate distribution shifts linearly over the stream, the fixed learning rate of SGD enables continuous adaptation. Batch IPF, applied once at the end, calibrates to the average composition over the collection period, which may not reflect the current population. Periodic IPF tracks the drift at discrete intervals. A rolling-window variant of batch IPF (recalibrating on the most recent n_0 observations) would also track drift, but requires choosing a window size. The streaming methods track drift continuously without this tuning choice, at the cost of not achieving exact calibration at any point.

4.3 When to Use What

The experiments suggest clear recommendations:

Batch IPF when the full sample is available before analysis and there is no need for interim estimates. It is the fastest, most accurate, and most familiar option.

Periodic IPF when interim estimates are needed at regular intervals (every 50–100 observations). It matches batch accuracy and is computationally cheap. This is the practical default for most streaming applications.

Streaming SGD (fixed η) when calibrated estimates are needed after every observation and the covariate distribution may drift. The fixed step size enables continuous adaptation but can produce weight oscillations.

Streaming SGD (Robbins-Monro) when weight stability matters more than exact marginal matching. The decaying step size acts as implicit regularization.

Streaming MWU when base weights are nonuniform (e.g., design weights from a complex sampling scheme). In this case, initialize each new observation's weight at its design weight u_{t+1} rather than one, and the multiplicative updates preserve the ratio structure of the base weights. MWU converges more slowly than SGD in our experiments but may be preferred when the multiplicative form is natural.

5 Discussion

5.1 Relation to IPF

IPF and the streaming algorithms solve the same feasibility problem through different decompositions. IPF processes one calibration variable at a time and scales every weight in a category by the same ratio t_j/m_j, zeroing out that variable's marginal error in a single pass. It converges by cycling through variables until all marginals are simultaneously satisfied. The streaming algorithms process one observation at a time and adjust its weight based on the gradient of the joint marginal loss across all variables. No single marginal error is zeroed out in one step; instead, all marginals improve simultaneously by a small amount.

The MWU update is multiplicative, like IPF, but the resemblance is structural rather than algorithmic. IPF multiplies by a ratio that depends only on the current marginal for one variable; MWU multiplies by an exponentiated gradient that depends on all marginals jointly. When the inner loop is run on all observations (not just the new one) and $K \to \infty$, both methods converge to weight vectors satisfying the marginal constraints on a feasible problem. They do not in general converge to the same weight vector: batch gradient descent on L finds the feasible point nearest the initialization in Euclidean distance (under normalization), while IPF finds the feasible point nearest the base weights in KL divergence.

5.2 Convergence behavior

On a fixed sample, the loss L can be analyzed by reparameterizing the weights so that $\sum_i w_i = n$. Under this normalization, the marginals become linear in $\mathbf{w}$, L is a convex quadratic, and projected gradient descent converges at the standard rate: $L(\mathbf{w}^{(T)}) - L^* \leq \|\mathbf{w}^{(0)} - \mathbf{w}^*\|_2^2/(2\eta T)$, giving marginal error $O(1/\sqrt{T})$ in the number of full gradient steps. This is a standard result from smooth convex optimization; the only problem-specific content is the Lipschitz constant $L_{\text{Lip}} = \|X\|_{\text{op}}^2/n^2 \leq p/n$ for binary indicators.

The streaming setting is harder to analyze formally. Two complications prevent direct application of the fixed-sample rate. First, the optimization problem changes with each arrival: at time t the algorithm minimizes $L^{(t)}$ over t weights; at time $t+1$ it minimizes $L^{(t+1)}$ over $t+1$ weights. Second, the algorithm updates only the new observation's weight, treating all previous weights as fixed. Standard convergence results assume a fixed objective and access to all coordinates.

What the experiments confirm is that the algorithms track the marginals reliably throughout the stream. Two properties explain why. Each new arrival perturbs the current marginals by $O(1/t)$ (one observation among t), so the K inner gradient steps need only compensate for a small perturbation rather than solve the problem from scratch. And the gradient (4) is computable from sufficient statistics, so it reflects the true marginal error at each step, not a stale estimate.

5.3 The early-vs-late asymmetry

The streaming algorithms exhibit an asymmetry that does not arise in batch calibration. Early observations arrive when the sample is small and the marginals are poorly estimated;

their weights are set under substantial uncertainty and are never revisited. Later observations arrive when the marginals are well-estimated, and their weights can be set with more precision. The quality of the final calibration therefore depends in part on the arrival order: adversarial or highly structured sequences could produce worse results than i.i.d. arrivals. In our i.i.d. simulations, this effect is mild. A natural mitigation would be to periodically revisit and re-optimize earlier weights, which would interpolate between the streaming and batch approaches at a tunable computational cost.

5.4 Squared-error versus KL calibration

The streaming algorithms minimize squared marginal error, not KL divergence. In the typical underdetermined case ($n \gg p$), many weight vectors satisfy the marginal constraints and the two objectives select different solutions. The squared-error solution tends toward weights closer to the initialization in Euclidean distance; the KL solution tends toward weights closer to the base weights in relative-entropy terms. Which is preferable depends on the application. When the solution is unique (up to scale), the distinction vanishes.

The relationship to entropy balancing deserves a note. Entropy balancing (Hainmueller, 2012) minimizes KL divergence from base weights subject to exact moment constraints, solving the problem via an unconstrained dual over Lagrange multipliers. Both IPF and entropy balancing target the same KL solution but by different computational routes: IPF cycles through constraints in the primal, entropy balancing solves for multipliers in the dual. Our streaming algorithms minimize a different objective and the multiplicative form of the MWU update arises from the exponentiated gradient rule, not from a KL geometry. In settings where the KL solution is preferred, one could replace (2) with the entropy balancing dual and apply the same streaming architecture, though the gradient computation would change.

5.5 Limitations and future work

The streaming algorithms update only the new observation's weight. If the target marginals are revised mid-stream (e.g., corrected administrative totals replace preliminary estimates, or a causal inference application switches to a different target population), previous observations' weights are stale and should be recalibrated. Periodic batch recalibration is the natural fix.

Future directions include adaptive step-size schedules (e.g., AdaGrad) that adjust to the local curvature of the marginal loss, extensions to continuous and multinomial calibration variables, multi-level post-stratification, and formal regret bounds for the growing-dimension streaming setting. A natural additional baseline, which we have not evaluated, is exponential-weighted moving average calibration: maintain an exponentially decayed running estimate of the weight adjustments and apply them to new observations. This would be $O(p)$ per observation with no inner loop but would lack the convergence guarantees of gradient-based methods. Comparison against such heuristics would sharpen the practical case for the methods developed here.

6 Conclusion

We have developed streaming algorithms for calibration weighting that provide calibrated weight estimates after each observation. The methods maintain running sufficient statistics and update only the new observation's weight, achieving per-observation cost of $O(Kp)$. Experiments show that streaming SGD closely approximates batch calibration accuracy, while periodic batch recalibration offers a practical middle ground. The framework applies wherever weights must be maintained incrementally to match known marginal distributions: rolling surveys, covariate balancing in sequential experiments, real-time dashboards, and online systems with distributional shift.

References

Arora, S., Hazan, E., and Kale, S. (2012). The multiplicative weights update method: A meta-algorithm and applications. *Theory of Computing*, 8(6):121–164.

Beck, A. and Teboulle, M. (2003). Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175.

Csiszár, I. (1975). I-divergence geometry of probability distributions and minimization problems. *The Annals of Probability*, 3(1):146–158.

Deming, W. E. and Stephan, F. F. (1940). On a least squares adjustment of a sampled frequency table when the expected marginal totals are known. *Annals of Mathematical Statistics*, 11(4):427–444.

Deville, J.-C. and Särndal, C.-E. (1992). Calibration estimators in survey sampling. *Journal of the American Statistical Association*, 87(418):376–382.

Deville, J.-C., Särndal, C.-E., and Sautory, O. (1993). Generalized raking procedures in survey sampling. *Journal of the American Statistical Association*, 88(423):1013–1020.

Gelman, A. and Little, T. C. (1997). Poststratification into many categories using hierarchical logistic regression. *Survey Methodology*, 23(2):127–135.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *International Conference on Machine Learning*.

Hainmueller, J. (2012). Entropy balancing for causal effects: A multivariate reweighting method to produce balanced samples in observational studies. *Political Analysis*, 20(1):25–46.

Hazan, E. (2016). *Introduction to Online Convex Optimization*, volume 2. Foundations and Trends in Optimization.

Imai, K. and Ratkovic, M. (2014). Covariate balancing propensity score. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 76(1):243–263.

Kolenikov, S. and Hammer, H. (2015). Simultaneous raking of survey weights at multiple levels. *Survey Methods: Insights from the Field*.

Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Technical report, Technical Report MSR-TR-1999-28, Microsoft Research.

Zadrozny, B. and Elkan, C. (2002). Transforming classifier scores into accurate multi-class probability estimates. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 694–699.

Zhao, Q. and Percival, D. (2017). Entropy balancing is doubly robust. *Journal of Causal Inference*, 5(1).

Zubizarreta, J. R. (2015). Stable weights that balance covariates for estimation with incomplete outcome data. *Journal of the American Statistical Association*, 110(511):910–922.